

services\analysis\cwe\catalog.py

```
"""
CWE Catalog Management
Handles loading and parsing CWE catalog from XML files
"""
# Operating system interface for file operations
import os
# XML parsing library for CWE catalog processing
import xml.etree.ElementTree as ET
# Type hints for return value structures
from typing import Dict, Any
# Application configuration for data directory paths
import config

class CWECatalogLoader:
    """Loads and manages CWE catalog data from XML"""

    def __init__(self):
        # Construct path to CWE catalog XML file using configuration
        self.catalog_path = os.path.join(config.DATA_DIR, 'CWE_Catalog.xml')
        # Initialize cache as None for lazy loading pattern
        self._catalog_cache = None

    def load_cwe_catalog(self) -> Dict[str, Dict[str, Any]]:
        """Load full CWE catalog from XML file"""
        # Return cached catalog if already loaded (performance optimization)
        if self._catalog_cache is not None:
            return self._catalog_cache

        # Initialize empty catalog for fallback scenarios
        catalog = {}

        # Check if catalog file exists before attempting to parse
        if not os.path.exists(self.catalog_path):
            print(f"[CWE Catalog] CWE catalog not found at {self.catalog_path}")
            self._catalog_cache = catalog
            return catalog

        try:
            print(f"[CWE Catalog] Loading CWE catalog from {self.catalog_path}")
            # Parse XML catalog file using ElementTree
            tree = ET.parse(self.catalog_path)
            root = tree.getroot()

            # Extract XML namespace for proper element querying
            namespace = self._extract_namespace(root)

            # Find all weakness entries in the XML catalog
            for weakness in root.findall(f'.//{namespace}Weakness'):
                # Parse individual weakness element into structured data
                cwe_entry = self._parse_weakness_element(weakness, namespace)
                if cwe_entry:
                    # Add parsed CWE to catalog using code as key
                    catalog[cwe_entry['code']] = cwe_entry

            print(f"[CWE Catalog] Loaded {len(catalog)} CWEs from catalog")

        except Exception as e:
            # Log error but continue with empty catalog for graceful degradation
            print(f"[CWE Catalog] Error loading CWE catalog: {e}")
```

```

# Cache parsed catalog for future requests
self._catalog_cache = catalog
return catalog

def _extract_namespace(self, root) -> str:
    """Extract namespace from XML root element"""
    # Check if root tag contains namespace information
    if root.tag.startswith('{'):
        # Extract namespace including closing brace for XPath compatibility
        return root.tag.split('}')[0] + '}'
    # Return empty string if no namespace present
    return ''

def _parse_weakness_element(self, weakness, namespace: str) -> Dict[str, Any]:
    """Parse a single weakness XML element"""
    # Extract CWE ID from XML element attributes
    cwe_id = weakness.get('ID')
    if not cwe_id:
        return None

    # Construct standard CWE code format
    cwe_code = f"CWE-{cwe_id}"
    # Get weakness name or use code as fallback
    name = weakness.get('Name', cwe_code)

    # Extract various types of weakness information using specialized methods
    description = self._extract_description(weakness, namespace)
    mitigations = self._extract_mitigations(weakness, namespace)
    examples = self._extract_examples(weakness, namespace)
    relationships = self._extract_relationships(weakness, namespace)

    # Build structured weakness data with fallback values
    return {
        'code': cwe_code,
        'name': name,
        'description': description or f'CWE-{cwe_id} weakness type',
        'mitigations': mitigations or ['Contact security team for specific mitigation strategies'],
        'examples': examples or [],
        'relationships': relationships or []
    }

def _extract_description(self, weakness, namespace: str) -> str:
    """Extract description from weakness element"""
    # Look for primary description element
    desc_elem = weakness.find(f'://{namespace}Description')
    if desc_elem is not None and desc_elem.text:
        return desc_elem.text.strip()

    # Try alternative extended description if primary not found
    extended_desc = weakness.find(f'://{namespace}Extended_Description')
    if extended_desc is not None and extended_desc.text:
        return extended_desc.text.strip()

    # Return empty string if no description found
    return ""

def _extract_mitigations(self, weakness, namespace: str) -> list:
    """Extract mitigation strategies from weakness element"""
    mitigations = []

    # Find all mitigation elements in weakness
    for mitigation_elem in weakness.findall(f'://{namespace}Mitigation'):

```

```

# Get development phase information if available
phase = mitigation_elem.get('Phase', '')
# Extract mitigation description text
description_elem = mitigation_elem.find(f'{{namespace}}Description')
if description_elem is not None and description_elem.text:
    mitigation_text = description_elem.text.strip()
# Add phase information to mitigation text for context
if phase:
    mitigation_text = f"[{phase}] {mitigation_text}"
mitigations.append(mitigation_text)

return mitigations

def _extract_examples(self, weakness, namespace: str) -> list:
    """Extract examples from weakness element"""
    examples = []

    # Look for demonstrative examples in weakness
    for example_elem in
    weakness.findall(f'././{{namespace}}Demonstrative_Example'):
        # Extract introductory text from examples
        intro_elem = example_elem.find(f'{{namespace}}Intro_Text')
        if intro_elem is not None and intro_elem.text:
            examples.append(intro_elem.text.strip())

    return examples

def _extract_relationships(self, weakness, namespace: str) -> list:
    """Extract relationships from weakness element"""
    relationships = []

    # Find related weakness elements
    for relation_elem in weakness.findall(f'././{{namespace}}Related_Weakness'):
        # Get relationship nature and target CWE ID
        nature = relation_elem.get('Nature', '')
        cwe_id = relation_elem.get('CWE_ID', '')
        if nature and cwe_id:
            # Build relationship tuple with standard CWE format
            relationships.append([f"CWE-{{cwe_id}}", nature])

    return relationships

def get_cwe_by_code(self, cwe_code: str) -> Dict[str, Any]:
    """Get specific CWE by code"""
    # Load catalog to ensure data availability
    catalog = self.load_cwe_catalog()
    # Return found CWE or fallback structure for missing CWEs
    return catalog.get(cwe_code, {
        'code': cwe_code,
        'name': f"CWE {{cwe_code}}",
        'description': f"Detailed information for {{cwe_code}} would be fetched from
        MITRE database.",
        'mitigations': ["Contact security team for specific mitigation
        strategies"],
        'examples': ["Examples would be loaded from MITRE database"],
        'relationships': []
    })

def search_cwes_by_keyword(self, keyword: str) -> list:
    """Search CWEs by keyword in name or description"""
    # Load full catalog for comprehensive searching
    catalog = self.load_cwe_catalog()
    # Convert keyword to lowercase for case-insensitive matching
    keyword_lower = keyword.lower()

```

```
matches = []
# Search through all CWEs in catalog
for cwe_code, cwe_data in catalog.items():
# Check if keyword appears in name or description (case-insensitive)
if (keyword_lower in cwe_data['name'].lower() or
keyword_lower in cwe_data['description'].lower()):
# Add matching CWE as tuple for easy unpacking
matches.append((cwe_code, cwe_data))

return matches
```